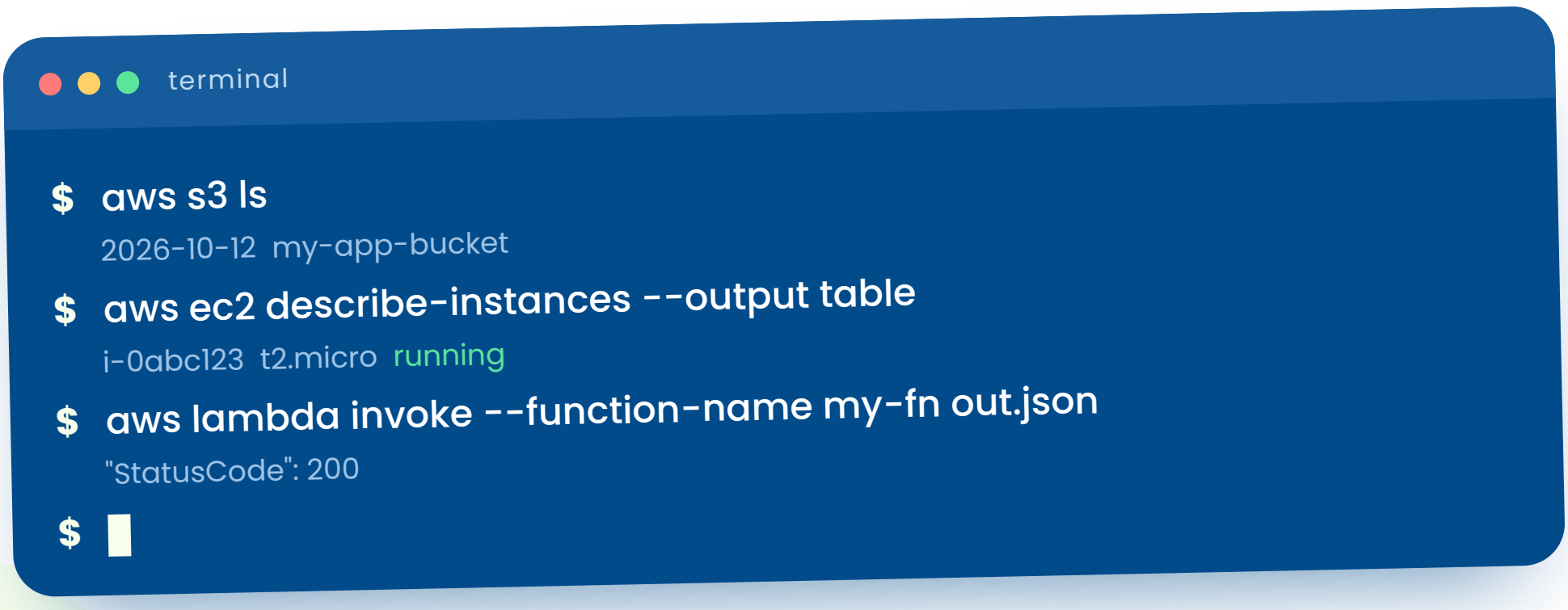
THE STUDY HANDBOOK

139 AWS CLI Commands

From your first **aws configure** to alarms, APIs and cost control.
Organised service by service.



EC2 · S3 · IAM · VPC · Security Groups · RDS · Lambda · API Gateway · DynamoDB · CloudWatch · CloudFormation

15	139	15
Topics	Must-know picks	Diagrams

Jaswindder Kummar
Follow for more insights on Cloud Strategy, DevOps, and Security Leadership.



Swipe →

START HERE / 11 COMMANDS

CLI Fundamentals

01

Install, configure and verify



INSTALL & VERIFY — 3

Install on macOS	`$ brew install awscli`
Check version	`$ aws --version`
Who am I?	`$ aws sts get-caller-identity`

CONFIGURE — 8

Interactive setup	`$ aws configure`
Create a profile	`$ aws configure --profile dev`
Use a profile	`$ aws s3 ls --profile dev`
List profiles	`$ aws configure list-profiles`
Show active config	`$ aws configure list`
Set default region	`$ aws configure set region ap-south-1`
Set output format	`$ aws configure set output json`
Help for a command	`$ aws s3 ls help`

TIP Settings live in **~/.aws/config** and keys in **~/.aws/credentials**.

Jaswindder Kummar
Follow for more insights on Cloud Strategy, DevOps, and Security Leadership.



Swipe →

COMPUTE / 11 COMMANDS

EC2 Instances

02

Launch, control and monitor servers



FIND INSTANCES 4

List all	`$ aws ec2 describe-instances`
Running only	`$ aws ec2 describe-instances --filters Name=instance-state-name,Values=running`
As a table	`$ aws ec2 describe-instances --output table`
Health status	`$ aws ec2 describe-instance-status --instance-ids i-123`

LIFECYCLE 7

Launch	`$ aws ec2 run-instances --image-id ami-xxx --instance-type t2.micro --key-name my-key`
Launch 2 at once	`$ aws ec2 run-instances --image-id ami-xxx --instance-type t2.micro --count 2`
Start	`$ aws ec2 start-instances --instance-ids i-123`
Stop	`$ aws ec2 stop-instances --instance-ids i-123`
Reboot	`$ aws ec2 reboot-instances --instance-ids i-123`
Terminate	`$ aws ec2 terminate-instances --instance-ids i-123`
Wait till running	`$ aws ec2 wait instance-running --instance-ids i-123`

TIP **Stopping** keeps the disk. **Terminating** deletes the instance for good.

Jaswindder Kummar
Follow for more insights on Cloud Strategy, DevOps, and Security Leadership.



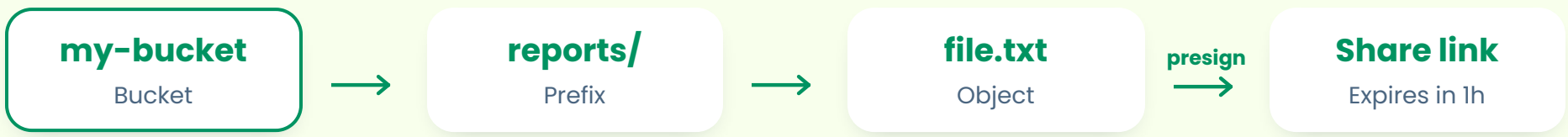
Swipe →

● STORAGE / 11 COMMANDS

S3 Buckets & Objects

03

Store, move and share files



BUCKETS 5

List buckets	`$ aws s3 ls`
Names only	`$ aws s3api list-buckets --query "Buckets[].Name" --output text`
Create in a region	`$ aws s3 mb s3://my-bucket --region ap-south-1`
Does it exist?	`$ aws s3api head-bucket --bucket my-bucket`
Delete with contents	`$ aws s3 rb s3://my-bucket --force`

OBJECTS 6

Upload	`$ aws s3 cp file.txt s3://my-bucket/`
Download	`$ aws s3 cp s3://my-bucket/file.txt .`
List everything	`$ aws s3 ls s3://my-bucket/ --recursive`
Mirror a folder	`$ aws s3 sync ./local s3://my-bucket/`
Delete all objects	`$ aws s3 rm s3://my-bucket/ --recursive`
Share for 1 hour	`$ aws s3 presign s3://my-bucket/file.txt --expires-in 3600`

TIP Use **s3** for quick file work and **s3api** for full control of settings.

Jaswindder Kummar
Follow for more insights on Cloud Strategy, DevOps, and Security Leadership.



Swipe →

IDENTITY / 10 COMMANDS

IAM Users & Groups

04

Who can sign in, and with which keys



USERS 5

List users	`$ aws iam list-users`
Names only	`$ aws iam list-users --query "Users[].UserName"`
Create user	`$ aws iam create-user --user-name devuser`
Rename user	`$ aws iam update-user --user-name devuser --new-user-name newuser`
User's policies	`$ aws iam list-attached-user-policies --user-name devuser`

ACCESS KEYS 3

Create key	`$ aws iam create-access-key --user-name devuser`
Key last used	`$ aws iam get-access-key-last-used --access-key-id AKIA...`
Disable key	`$ aws iam update-access-key --access-key-id AKIA... --status Inactive`

GROUPS 2

Create group	`$ aws iam create-group --group-name developers`
Add user	`$ aws iam add-user-to-group --user-name devuser --group-name developers`

TIP Attach policies to **groups**, not to individual users. It scales better.

Jaswindder Kummar
Follow for more insights on Cloud Strategy, DevOps, and Security Leadership.



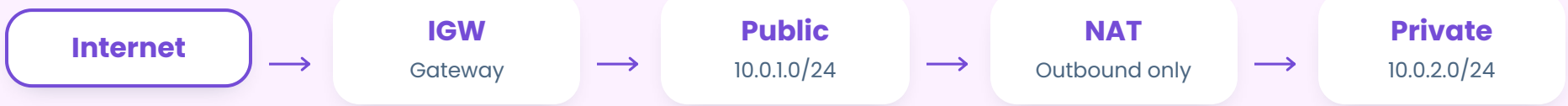
Swipe →

NETWORK / 10 COMMANDS

VPC & Networking

05

Build your own private network



VPC & SUBNETS

4

Create VPC

```
$ aws ec2 create-vpc --cidr-block 10.0.0.0/16
```

DNS hostnames on

```
$ aws ec2 modify-vpc-attribute --vpc-id vpc-123 --enable-dns-hostnames
```

Create subnet

```
$ aws ec2 create-subnet --vpc-id vpc-123 --cidr-block 10.0.1.0/24
```

Auto public IP

```
$ aws ec2 modify-subnet-attribute --subnet-id subnet-123
--map-public-ip-on-launch
```

GATEWAYS & ROUTES

6

Create IGW

```
$ aws ec2 create-internet-gateway
```

Attach IGW

```
$ aws ec2 attach-internet-gateway --internet-gateway-id igw-123 --vpc-id
vpc-123
```

Route table

```
$ aws ec2 create-route-table --vpc-id vpc-123
```

Route to internet

```
$ aws ec2 create-route --route-table-id rtb-123 --destination-cidr-block 0.0.0.0/0
--gateway-id igw-123
```

Link subnet

```
$ aws ec2 associate-route-table --subnet-id subnet-123 --route-table-id rtb-123
```

Elastic IP

```
$ aws ec2 allocate-address --domain vpc
```

TIP **Public** subnets route to the IGW. **Private** subnets reach out through NAT.

Jaswindder Kummar
Follow for more insights on Cloud Strategy, DevOps, and Security Leadership.



Swipe →

FIREWALL / 8 COMMANDS

Security Groups

06

Stateful firewall for EC2, RDS and more



MANAGE GROUPS — 3

List groups

```
$ aws ec2 describe-security-groups
```

Create group

```
$ aws ec2 create-security-group --group-name web-sg --description "Web"
```

Find by name

```
$ aws ec2 describe-security-groups --filters Name=group-name,Values=web-sg
```

OPEN PORTS (INBOUND) — 3

SSH from you

```
$ aws ec2 authorize-security-group-ingress --group-id sg-123 --protocol tcp
  --port 22 --cidr <your-ip>/32
```

HTTPS

```
$ aws ec2 authorize-security-group-ingress --group-id sg-123 --protocol tcp
  --port 443 --cidr 0.0.0.0/0
```

MySQL in VPC

```
$ aws ec2 authorize-security-group-ingress --group-id sg-123 --protocol tcp
  --port 3306 --cidr 10.0.0.0/16
```

REVIEW & REMOVE — 2

Inbound rules

```
$ aws ec2 describe-security-group-rules --filters Name=security-group-rule-
  type,Values=ingress
```

Close a port

```
$ aws ec2 revoke-security-group-ingress --group-id sg-123 --protocol tcp --port
  80 --cidr 0.0.0.0/0
```

TIP Lock SSH to your own IP (**/32**). Never leave port 22 open to **0.0.0.0/0**.

Jaswindder Kummar
Follow for more insights on Cloud Strategy, DevOps, and Security Leadership.



Swipe →

DATABASE / 8 COMMANDS

07

RDS Databases

Managed databases, backups and replicas



INSTANCES 4

List databases

```
$ aws rds describe-db-instances
```

Stop a database

```
$ aws rds stop-db-instance --db-instance-identifier mydb
```

Resize

```
$ aws rds modify-db-instance --db-instance-identifier mydb
  --db-instance-class db.t3.small
```

Multi-AZ on

```
$ aws rds modify-db-instance --db-instance-identifier mydb --multi-az
  --apply-immediately
```

BACKUP & RECOVERY 4

7-day backups

```
$ aws rds modify-db-instance --db-instance-identifier mydb
  --backup-retention-period 7 --apply-immediately
```

Take snapshot

```
$ aws rds create-db-snapshot --db-snapshot-identifier my-snap
  --db-instance-identifier mydb
```

Restore snapshot

```
$ aws rds restore-db-instance-from-db-snapshot --db-instance-identifier
  mydb-restore --db-snapshot-identifier my-snap
```

Read replica

```
$ aws rds create-db-instance-read-replica --db-instance-identifier mydb-rr
  --source-db-instance-identifier mydb
```

TIP Use **--skip-final-snapshot** only in test. In prod, keep a final snapshot.

Jaswindder Kummar
Follow for more insights on Cloud Strategy, DevOps, and Security Leadership.



Swipe →

SERVERLESS / 9 COMMANDS

Lambda Functions

08

Serverless code from deploy to invoke



DEPLOY 5

List functions	`$ aws lambda list-functions --query "Functions[].FunctionName"`
Update code	`$ aws lambda update-function-code --function-name my-fn --zip-file fileb://fn.zip`
Set env vars	`$ aws lambda update-function-configuration --function-name my-fn --environment "Variables={KEY=VALUE}"`
Publish version	`$ aws lambda publish-version --function-name my-fn`
Create alias	`$ aws lambda create-alias --function-name my-fn --name prod --function-version 1`

RUN & MONITOR 4

Invoke	`$ aws lambda invoke --function-name my-fn out.json`
With payload	`$ aws lambda invoke --function-name my-fn --payload fileb://input.json out.json`
Cap concurrency	`$ aws lambda put-function-concurrency --function-name my-fn --reserved-concurrent-executions 5`
Read logs	`$ aws logs filter-log-events --log-group-name /aws/lambda/my-fn`

TIP Point clients at an alias like **prod**, then shift it to a new version.

Jaswindder Kummar
Follow for more insights on Cloud Strategy, DevOps, and Security Leadership.



Swipe →

APIS / 8 COMMANDS

API Gateway

09

A REST API in front of Lambda



BUILD IT, STEP BY STEP

8

1 Create API

```
$ aws apigateway create-rest-api --name my-api
```

2 Find root id

```
$ aws apigateway get-resources --rest-api-id abc123
```

3 Add /users

```
$ aws apigateway create-resource --rest-api-id abc123 --parent-id <root>
  --path-part users
```

4 Add GET

```
$ aws apigateway put-method --rest-api-id abc123 --resource-id <res>
  --http-method GET --authorization-type NONE
```

5 Wire Lambda

```
$ aws apigateway put-integration --rest-api-id abc123 --resource-id <res>
  --http-method GET --type AWS_PROXY --integration-http-method POST --uri
  <lambda-uri>
```

6 Allow invoke

```
$ aws lambda add-permission --function-name my-fn --statement-id apigw
  --action lambda:InvokeFunction --principal apigateway.amazonaws.com
```

7 Test

```
$ aws apigateway test-invoke-method --rest-api-id abc123 --resource-id <res>
  --http-method GET
```

8 Deploy

```
$ aws apigateway create-deployment --rest-api-id abc123 --stage-name prod
```

TIP Lambda needs **add-permission** so API Gateway is allowed to invoke it.

Jaswindder Kummar
Follow for more insights on Cloud Strategy, DevOps, and Security Leadership.



Swipe →

STORAGE / 8 COMMANDS

10

S3 Security & Lifecycle

Lock down and automate your buckets

Block public Access block → **Policy** Who can access → **Encryption** AES-256 → **Versioning** Undo deletes → **Lifecycle** Auto-archive

PROTECT 5

Block public access

```
$ aws s3api put-public-access-block --bucket my-bucket --public-access-block-configuration BlockPublicAcls=true,IgnorePublicAcls=true,BlockPublicPolicy=true,RestrictPublicBuckets=true
```

Apply policy

```
$ aws s3api put-bucket-policy --bucket my-bucket --policy file://policy.json
```

View policy

```
$ aws s3api get-bucket-policy --bucket my-bucket
```

Encrypt by default

```
$ aws s3api put-bucket-encryption --bucket my-bucket --server-side-encryption-configuration file://sse.json
```

Versioning on

```
$ aws s3api put-bucket-versioning --bucket my-bucket --versioning-configuration Status=Enabled
```

AUTOMATE 3

Lifecycle rules

```
$ aws s3api put-bucket-lifecycle-configuration --bucket my-bucket --lifecycle-configuration file://rules.json
```

View rules

```
$ aws s3api get-bucket-lifecycle-configuration --bucket my-bucket
```

Remove rules

```
$ aws s3api delete-bucket-lifecycle --bucket my-bucket
```

TIP Turn on **Block Public Access** first, then open only what you need.

Jaswindder Kummar
Follow for more insights on Cloud Strategy, DevOps, and Security Leadership.



Swipe →

NOSQL / 9 COMMANDS

DynamoDB

11

Fast NoSQL tables and items

MyTable (Table) keyed by **id** (Partition key) stores **Items** (JSON-like) + index **email-index** (GSI)

TABLES 3

List tables

```
$ aws dynamodb list-tables
```

Table details

```
$ aws dynamodb describe-table --table-name MyTable
```

Create table

```
$ aws dynamodb create-table --table-name MyTable --attribute-definitions
AttributeName=id,AttributeType=S --key-schema
AttributeName=id,KeyType=HASH --billing-mode PAY_PER_REQUEST
```

ITEMS 4

Add item

```
$ aws dynamodb put-item --table-name MyTable --item file://item.json
```

Get by key

```
$ aws dynamodb get-item --table-name MyTable --key '{"id":{"S":"1"}}'
```

Query by key

```
$ aws dynamodb query --table-name MyTable --key-condition-expression "id =
:id" --expression-attribute-values file://v.json
```

Scan all

```
$ aws dynamodb scan --table-name MyTable
```

PROTECT 2

Create backup

```
$ aws dynamodb create-backup --table-name MyTable --backup-name
MyBackup
```

PITR on

```
$ aws dynamodb update-continuous-backups --table-name MyTable --point-in-
time-recovery-specification PointInTimeRecoveryEnabled=true
```

TIP **Query** reads by key and is cheap. **Scan** reads the whole table, so use it sparingly.

Jaswindder Kummar
Follow for more insights on Cloud Strategy, DevOps, and Security Leadership.



Swipe →

OBSERVABILITY / 8 COMMANDS

CloudWatch

12

Logs, metrics, alarms and dashboards



LOGS 3

List log groups

```
$ aws logs describe-log-groups
```

Find errors

```
$ aws logs filter-log-events --log-group-name MyLogGroup --filter-pattern ERROR
```

Count errors

```
$ aws logs put-metric-filter --log-group-name MyLogGroup --filter-name Errors
--filter-pattern ERROR --metric-transformations
metricName=ErrorCount,metricNamespace=MyApp,metricValue=1
```

METRICS & ALARMS 5

List EC2 metrics

```
$ aws cloudwatch list-metrics --namespace AWS/EC2
```

CPU alarm

```
$ aws cloudwatch put-metric-alarm --alarm-name HighCPU --namespace
AWS/EC2 --metric-name CPUUtilization --statistic Average --period 300
--threshold 70 --comparison-operator GreaterThanThreshold
--evaluation-periods 2
```

Test alarm

```
$ aws cloudwatch set-alarm-state --alarm-name HighCPU --state-value ALARM
--state-reason Test
```

Mute alarm

```
$ aws cloudwatch disable-alarm-actions --alarm-names HighCPU
```

Dashboard

```
$ aws cloudwatch put-dashboard --dashboard-name Ops --dashboard-body
file://dash.json
```

TIP Test an alarm safely with **set-alarm-state** before you rely on it.

Jaswindder Kummar
Follow for more insights on Cloud Strategy, DevOps, and Security Leadership.



Swipe →

IDENTITY / 9 COMMANDS

13

IAM Roles & Policies

Permissions for people and services

1 · User create-user → **2 · Group** create-group → **3 · Join** add-user-to-group → **4 · Policy** attach-group-policy

POLICIES 4

Your own policies	`$ aws iam list-policies --scope Local`
Create policy	`$ aws iam create-policy --policy-name mypolicy --policy-document file://policy.json`
New version	`$ aws iam create-policy-version --policy-arn <arn> --policy-document file://policy.json --set-as-default`
Attach to group	`$ aws iam attach-group-policy --group-name developers --policy-arn <arn>`

ROLES 3

List roles	`$ aws iam list-roles`
Create role	`$ aws iam create-role --role-name myrole --assume-role-policy-document file://trust.json`
Attach policy	`$ aws iam attach-role-policy --role-name myrole --policy-arn <arn>`

ACCOUNT 2

User's groups	`$ aws iam list-groups-for-user --user-name myuser`
Password rules	`$ aws iam update-account-password-policy --minimum-password-length 12`

TIP **Least privilege**: grant only the actions a user or role really needs.

Jaswindder Kummar
Follow for more insights on Cloud Strategy, DevOps, and Security Leadership.



Swipe →

COMPUTE / 10 COMMANDS

EC2 Toolkit

14

Keys, images, volumes and IPs



ACCESS 3

Create key pair
```
$ aws ec2 create-key-pair --key-name MyKey --query KeyMaterial --output text > MyKey.pem
```

Secure the key
```
$ chmod 400 MyKey.pem
```

Import public key
```
$ aws ec2 import-key-pair --key-name MyKey --public-key-material fileb://my-key.pub
```

IMAGES & STORAGE 5

Amazon AMIs
```
$ aws ec2 describe-images --owners amazon
```

Build an AMI
```
$ aws ec2 create-image --instance-id i-123 --name MyImage
```

Retire an AMI
```
$ aws ec2 deregister-image --image-id ami-123
```

List volumes
```
$ aws ec2 describe-volumes
```

Snapshot volume
```
$ aws ec2 create-snapshot --volume-id vol-123 --description "Backup"
```

IPS & REGIONS 2

Attach Elastic IP
```
$ aws ec2 associate-address --instance-id i-123 --allocation-id eipalloc-123
```

List regions
```
$ aws ec2 describe-regions
```

TIP Run **chmod 400** on your .pem key, or SSH will refuse to use it.

Jaswindder Kummar
Follow for more insights on Cloud Strategy, DevOps, and Security Leadership.



Swipe →

● LEVEL UP / 9 COMMANDS

15

Pro Tips & Cost Control

Work faster and keep the bill low

Command describe-... → **Filter** --query → **Format** json · table · text → **Save** Watch costs

SHAPE YOUR OUTPUT 4

Readable table

```
$ aws ec2 describe-instances --output table
```

Just the IDs

```
$ aws ec2 describe-instances --query "Reservations[].Instances[].InstanceId"
```

Skip terminated

```
$ aws ec2 describe-instances --query "Reservations[].Instances[?
State.Name!='terminated'].InstanceId" --output text
```

Help anywhere

```
$ aws <service> <command> help
```

INFRA & COST 5

List stacks

```
$ aws cloudformation list-stacks
```

Create stack

```
$ aws cloudformation create-stack --stack-name MyStack --template-body
file://template.yaml
```

Bucket size

```
$ aws s3 ls s3://my-bucket --recursive --summarize --human-readable
```

Monthly cost

```
$ aws ce get-cost-and-usage --time-period Start=2026-09-01,End=2026-10-01
--granularity MONTHLY --metrics BlendedCost
```

Your budgets

```
$ aws budgets describe-budgets --account-id 123456789012
```

TIP Add **--dry-run** to EC2 commands to test permissions without changing anything.

Jaswindder Kummar
Follow for more insights on Cloud Strategy, DevOps, and Security Leadership.



Swipe →

QUICK REVISION

Keep learning.
Keep building.
Keep growing.

Explore → Practise → Build → Grow

Configure first 01
Run **aws configure** before anything else.

Use profiles 02
--profile and **--region** for many accounts.

Query smart 03
--query keeps only what you need.

Pick the format 04
table to read, **json** for scripts.

Least privilege 05
Grant IAM only what's needed.

Careful deletes 06
terminate and **rm** can't be undone.

Save for revision

Share with a friend

Follow for more

YOUR GUIDE ON THIS JOURNEY

Jaswindder Kummar

Follow for more insights on Cloud Strategy, DevOps, and Security Leadership.

